Premier League Match Probabilities

A statistical model that estimates win, draw and loss probabilities for English Premier League matches, evaluated honestly against the sharpest benchmark available: closing betting-market odds. Predictions for upcoming matches are published to this repository before kickoff, so the model's live track record can be checked by anyone.

The project covers the full pipeline: collecting and validating 30+ seasons of data from four sources, building and tuning a Poisson goals model with expected goals (xG), testing it with walk-forward backtesting on seasons it never saw during development, and serving it through a web app with an automated, tested weekly update.

Tech: Python, SQL (SQLite), pandas, NumPy, SciPy, FastAPI, Jinja, pytest, GitHub Actions

Results

All numbers below are from **held-out seasons (2021/22 to 2025/26, 1,921 matches)** that played no part in choosing the model's settings. Lower log loss is better; guessing one-third for every outcome scores 1.0986.

Model	Log loss
Goals only, initial settings	0.9824
Goals only, tuned	0.9813
+ expected goals (xG) blend	0.9778
+ xG with re-tuned recency weighting (**current model**)	**0.9754**
Closing market (Pinnacle / Betfair Exchange, margin removed)	0.9592

Adding xG and re-tuning closed **27% of the gap** between the goals-only model and the closing market. Put another way, the current model gives the actual result a typical probability of **37.7%**, against the market's **38.3%**.

The model does not beat the market, and isn't expected to: closing odds reflect injuries, confirmed lineups and professional bettors' information, while this model only sees past results. The goal is to measure how close a transparent, results-based model can get, and which changes actually help.

Live track record

Since October 2026, every prediction for an upcoming match is committed to `records/live_predictions.csv` before kickoff by an automated job, and never changed afterwards; the Git history shows when each one was made. The website's Track record page scores these predictions, and only these, once the matches are played. A few rounds is far too small a sample to judge the model by, so the backtest above is the main evidence until the record grows.

Data

Source	Coverage	Contents
football-data.co.uk	1993/94 onward	Results, half-time scores, match stats (from 2000/01), 1X2 / over-under / Asian handicap odds from 20+ bookmakers, pre-match and closing; upcoming fixtures
Understat	2014/15 onward	Expected goals (xG) per team per match; results before football-data.co.uk posts them
Fantasy Premier League	2016/17 onward	Player prices (via the FPL Historical Dataset), current injuries and availability
transfermarkt-datasets	2013/14 to June 2026	Player market valuations, used in the squad-value experiments

The database holds 13,000+ Premier League matches. Some of the data-quality work involved:

- **Inconsistent file formats across 30+ seasons.** Column names change over time (e.g. `BbAv` became `Avg`), and some seasons have rows with more fields than the header. The importer handles both, and every original row is also stored as JSON so no source column is ever lost.
- **Unmapped column report.** Each import lists any source columns not mapped to a table, which is how new bookmakers and a new xG column were discovered.
- **Cross-source validation.** Each source spells team names differently (e.g. "Manchester United" vs "Man United"), resolved through an alias table. All 4,610 matches linked between Understat and football-data.co.uk were checked against both sources' final scores, with 100% agreement.
- **Source changes and delays.** Midway through development, Understat stopped embedding data in its pages; the importer failed loudly rather than importing nothing, and the fix was isolated to one function. football-data.co.uk can take days to post results, so results are filled in from Understat in the meantime and confirmed when football-data.co.uk catches up, without creating duplicate matches.
- **No look-ahead.** Squad values for any week are built only from information available by that date, so a backtest never benefits from knowing, for example, a player's later transfer.

Model

Each team has an **attack** and a **defense** rating, plus a league-wide **home advantage**. A team's expected goals in a match are:

```
expected goals = base rate × home advantage (if at home) × attack ÷ opponent's defense
```

Each team's goals are modeled as a Poisson distribution around its expected goals, which gives the probability of every exact scoreline. From those come home/draw/away probabilities, over/under totals, both teams to score, and the likeliest scores.

Ratings are fitted by **maximum likelihood** (SciPy's L-BFGS-B with an analytic gradient, about 30× faster than numerical differentiation) using the last three years of matches, with:

- **Recency weighting:** a match's weight halves every 180 days.
- **Shrinkage (L2 regularization):** ratings are pulled toward average, so teams with few matches, such as newly promoted sides, don't get extreme ratings from a handful of results.
- **xG blend:** ratings are learned from a 50/50 blend of actual goals and xG.

A **season simulator** plays out the rest of the season 10,000 times with the model to estimate each team's chances of winning the title, finishing in the top four, or being relegated.

Evaluation

- **Walk-forward backtest:** for every week since 2016/17, the model is trained only on matches played before that week, then predicts that week's games. It never sees a result it is predicting.
- **Validation / test split:** all settings were tuned on 2016/17–2020/21, and final results are reported on 2021/22–2025/26.
- **Metrics:** log loss and Brier score, compared against closing odds from Pinnacle (Betfair Exchange where Pinnacle is unavailable), with the bookmaker margin removed.

Findings

- **The gap to the market is spread evenly.** An error analysis split the backtest by team, stage of the season, favorite strength and outcome. The model is well calibrated overall (slightly too cautious on strong favorites), and no single group accounts for the shortfall. That pattern points to missing information, mainly team news, rather than a flaw in the math.
- **Squad value helps early in the season, and only then.** See below.
- **Promoted teams aren't a weak spot.** A dedicated check found the model's error on newly promoted teams' matches only slightly worse than usual, so importing second-division data to rate them wasn't worth the added complexity.
- **Home advantage is shrinking.** Home teams won around 45–50% of matches in the 1990s and 2000s, and closer to 41–48% recently.
- **Crowds matter.** In 2020/21, played without fans, away teams won more often than home teams (40.3% vs 37.9%), the only season where that happened.
- **Scoring jumped in 2023/24** to 3.28 goals per game, the highest on record, coinciding with a crackdown on time-wasting that added stoppage time. The goals-only model was slow to adapt that season.
- **Goals and xG work best together.** On validation seasons, goals only scored 0.9571, a 50/50 blend 0.9532, and pure xG 0.9552. Actual goals carry information xG misses, such as finishing and goalkeeping quality.
- **xG allows a shorter memory.** With xG, the best recency half-life dropped from 365 to 180 days, and less shrinkage was needed.
- **Dixon–Coles didn't help.** This well-known correction for low-scoring draws made predictions slightly worse: the model already predicted slightly more draws than occurred (23.7% vs 22.5%). The apparent excess of low scores in the raw data came from pooling strong and weak teams, which team-specific ratings already account for.

Squad value

Pulling ratings toward each squad's value (FPL prices or Transfermarkt valuations) improved early-season predictions but made later ones slightly worse, since by then results already show how good a team is. Making its influence fade as the season goes on (halving every 12 rounds) kept nearly all of the early gain. Log loss on the 1,873 held-out matches with squad values available:

	Current model	Squad value, full strength	Squad value, fading	Market
All matches	0.9735	0.9737	**0.9718**	0.9570
Rounds 1–10	0.9558	**0.9492**	0.9500	0.9473
Rounds 11–38	0.9800	0.9826	**0.9797**	0.9605

The fading version closes 11% of the overall gap to the market, and about two-thirds of the gap in rounds 1–10. One caveat: on the tuning seasons the full-strength version scored slightly better, so the held-out seasons decided between two finalists. The fading version is preferred because, by design, it can't cause the late-season loss. It isn't used for live predictions yet.

Website

A FastAPI app with Jinja templates, currently run locally:

- **Upcoming:** next fixtures with model probabilities next to the bookmakers' odds
- **Match pages:** exact-score grid, over/under (with the market's view), both teams to score
- **Results** by week and **league tables** for every season since 1993/94, with team pages
- **Forecast:** season simulation with title, top-four and relegation chances
- **Track record:** live predictions only, scored against results and the closing market
- **Backtest** and **Ratings:** held-out performance and current team ratings

`build_site.py` can also render the whole site as static HTML for GitHub Pages.

Automation and testing

- **One command updates everything:** `update.py` runs the results, xG, backtest, fixtures, prediction and availability steps in order, stopping at the first failure.
- **GitHub Actions** runs the test suite on every push, and twice a week runs the update and commits new predictions to `records/` before kickoff.
- **73 pytest tests** cover the importers (including format quirks and source changes), the model (valid probabilities, recovering known team strengths from simulated leagues, no look-ahead in the backtest), the prediction record (nothing changes after kickoff), the simulator and the website. Tests use simulated leagues and temporary databases, never real data or the network.

Project structure

```
update.py               Runs the weekly update, step by step
import_epl.py           football-data.co.uk results, stats and odds (cached, safe to re-run)
import_understat.py     Understat xG; fills in results before football-data.co.uk posts them
import_fixtures.py      Upcoming fixtures and pre-match odds
model.py                The model, backtest and scoring
run_backtest.py         Saves walk-forward backtest predictions
predict_upcoming.py     Predicts upcoming matches and records them before kickoff
records.py              The official prediction record in records/
simulate.py             Monte Carlo season simulator
fetch_availability.py   Daily snapshots of player injuries and availability
app.py, templates/      The website (build_site.py renders it as static HTML)
error_analysis.py       Where the model loses ground to the market
promoted_teams.py       How the model handles newly promoted teams
import_fpl_values.py, import_transfermarkt.py, tune_squad_value.py, tune_value_fade.py
                        Squad-value data and experiments
schema.sql              Database schema, designed to support more leagues and sports
team_aliases.sql        Team-name mappings between data sources
checks.sql              SQL sanity checks and data exploration queries
poisson_model.ipynb     Model development notebook
tests/                  pytest suite
```

Running it

Requires Python 3.11+.

```
git clone https://github.com/emrozin/epl-probabilities.git
cd epl-probabilities
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

python update.py              # builds data/sports.db (the first run downloads every season)
uvicorn app:app --reload      # then open http://127.0.0.1:8000
pytest                        # run the tests
```

Roadmap

- More sports on the same site (NHL and NBA), chosen from a menu
- Player availability: adjust for injured and suspended players, the largest remaining source of the gap to the market
- An optional "model + market" forecast that blends the model with pre-match odds
- Use the fading squad-value adjustment for live predictions

Acknowledgements

Data comes from football-data.co.uk, Understat and Fantasy Premier League; please respect their terms of use. Historical FPL prices come from the FPL Historical Dataset by Vaastav Anand (MIT License), and player valuations from transfermarkt-datasets (CC0). Understat has no official API, so its importer may need updating if the site changes. This is a statistics project, not betting advice.